

STL PRACTICE TASKS

Task 1. Vector Student Marks Manager

Create a program that:

- Takes n student marks.
- Stores them in a `vector<int>`.
- Prints all marks using:
 - normal indexing
 - iterator
 - range-based loop
- Adds 3 more marks using `push_back()`.
- Removes the last mark.
- Prints:
 - size
 - capacity
 - first element
 - last element
- Find the highest and lowest marks **without using** `sort()`.

STL: `vector`, `iterators`, `push_back`, `pop_back`, `size`, `capacity`

Task 2. Vector + Iterator Remove Even Numbers

Given:

10 15 20 25 30 35 40

Remove all even numbers from the vector **using iterators**.

Expected:

15 25 35

Don't use a second vector.

STL: `vector`, `iterators`, `erase()`

Task 3. List Train Coach System

Represent train coaches using:

`list<string>`

Start with:

A B C D E

Perform:

1. Add coach x at the beginning.
2. Add coach y at the end.
3. Find coach c .
4. Insert z before c .
5. Remove coach d .
6. Print the final train.

Important: Use an iterator to locate c .

This gives you an actual reason to use `list` instead of `vector`.

Task 4. Deque Browser History

Create a browser-history system using:

```
deque<string>
```

Operations:

1. Visit new website
2. Go back
3. Go forward
4. Show current history

For example:

```
Google  
YouTube  
GitHub  
LeetCode
```

Your program should demonstrate inserting/removing from **both ends**.

STL: `deque`, `push_front`, `push_back`, `pop_front`, `pop_back`

Level 2 Small STL Combinations

5. Pair — Coordinate System

Store coordinates using:

```
pair<int, int>
```

Take n points from the user:

```
(2, 3)
(5, 1)
(7, 8)
(1, 4)
```

Then:

- Print all points.
- Find the point closest to the origin.
- Find the point with the largest x .
- Swap two points using `swap()`.

Don't worry about advanced mathematics the point is practicing `pair`.

Task 6. Pair + Vector Student Record

Represent every student as:

```
pair<string, int>
```

Example:

```
Ali 85
Ahmed 72
Usman 91
Hamza 67
```

Store them in:

```
vector<pair<string, int>>
```

Then:

- Add students.
- Print all students.
- Find the student with highest marks.
- Find a student by name.
- Update a student's marks.

This is a **very important STL pattern** because you'll frequently encounter:

```
vector<pair<...>>
```

Task 7. Push vs Emplace Product Inventory

Create:

```
vector<pair<string, int>>
```

Add products using both:

```
push_back()
```

and

```
emplace_back()
```

For example:

```
Laptop 120000  
Mouse 2500  
Keyboard 5000
```

Your task is simply to implement the same inventory twice:

- once using `push_back`
- once using `emplace_back`

Then explain in comments **what difference you observe in how the object/pair is constructed.**

Level 3 Stack & Queue

Task 8. Stack — Undo System

Create a simple text editor simulation.

Commands:

1. Type something
2. Undo
3. Show current text

Example:

```
Type: Hello  
Type: World  
Undo
```

Output:

```
Hello
```

Use:

```
stack<string>
```

Think about **what should be pushed and what should be popped**.

Task 9. Stack Balanced Parentheses

Given:

```
(( ( ) ) )
```

Determine whether parentheses are balanced.

Test:

```
(( ( ) ) )  
( ) ( ) ( )  
( ( ) )  
(( ( ) )
```

Use:

```
stack<char>
```

This is your first problem where the STL container actually helps you solve an algorithmic problem.

Task 10. Queue Customer Service System

Create a customer queue.

Operations:

1. Add customer
2. Serve customer
3. Show next customer
4. Show all waiting customers

Example:

```
Ali  
Ahmed  
Usman
```

Serving should happen in:

```
Ali → Ahmed → Usman
```

STL: `queue`

11. Deque vs Queue

Implement the same customer system using a deque.

Now allow:

```
VIP customer → front
Normal customer → back
Serve → front
```

Example:

```
Normal: Ali
Normal: Ahmed
VIP: Hamza
```

Queue becomes:

```
Hamza → Ali → Ahmed
```

This should make the **reason for deque vs queue** much clearer.

Level 4 Priority Queue

Task 12. Hospital Emergency System

Create:

```
priority_queue<pair<int, string>>
```

where priority is:

```
1 = Normal
2 = Serious
3 = Critical
```

Patients:

```
Ali      1
Ahmed    3
Usman    2
Hamza    3
```

When treating patients, they should come out according to priority.

Expected order:

Ahmed
Hamza
Usman
Ali

This is a very good priority queue exercise.

Task 13. Top 3 Largest Numbers

Given:

10 4 25 7 19 30 15

Use a `priority_queue` to find the **three largest numbers**.

Don't use `sort()`.

Expected:

30 25 19

Level 5 Map / Multimap / Unordered Map

Task 14. Map Student Marks Database

Create:

```
map<string, int>
```

Store:

Ali -> 85
Ahmed -> 72
Usman -> 91
Hamza -> 67

Implement:

1. Add student
2. Search student
3. Update marks
4. Delete student
5. Display all students

Then observe the order in which students are displayed.

Task 15. Map Frequency Counter

Given:

```
apple banana apple orange banana apple
```

Create a frequency table:

```
apple -> 3  
banana -> 2  
orange -> 1
```

Use:

```
map<string, int>
```

This is one of the **most important uses of maps**.

Task 16. Unordered Map Same Frequency Problem

Implement exactly the same frequency counter using:

```
unordered_map<string, int>
```

Then compare the output/order with `map`.

Your goal here isn't just to know the syntax.

You should be able to answer:

When would I choose `map` and when would I choose `unordered_map`?

Task 17. Multimap — Student → Multiple Subjects

Suppose:

```
Ali -> Math  
Ali -> Physics  
Ali -> Programming
```

```
Ahmed -> Math  
Ahmed -> Chemistry
```

Use:

```
multimap<string, string>
```

Then:

- Add subjects.
- Display all subjects of Ali.
- Display all students.
- Remove a particular entry.

This gives you a practical reason why `multimap` exists.

Level 6 Set

18. Remove Duplicates

Given:

10 20 10 30 20 40 10 50

Use:

`set<int>`

to produce:

10 20 30 40 50

Then notice the **automatic sorting**.

19. Set Common Elements

Given:

A = {1, 2, 3, 4, 5}

B = {3, 4, 5, 6, 7}

Find:

3 4 5

using sets.

Try to solve it using set operations rather than manually checking every pair.

20. Mini Project Student Management System (OPTIONAL)

Build a console program that uses **multiple STL containers**.

Each student has:

ID
Name
Marks
Subjects

Your program should support:

1. Add student
2. Remove student
3. Search student
4. Update marks
5. Display all students
6. Find highest scorer
7. Add subject to student
8. Display unique subjects
9. Add students to an admission queue
10. Process students according to priority

You decide which STL container is appropriate.

For example, you might end up with something like:

```
map<int, Student>
```

and inside Student:

```
vector<string>  
set<string>
```

and for admission:

```
queue<int>
```

and priority processing:

```
priority_queue<...>
```

Don't copy a predefined solution. The important part of this exercise is deciding:

"Which STL container should I use here, and why?"